English–Hindi Neural Machine Translation via Transformer with Optuna Hyperparameter Tuning

Assignment 4 — M25CSA027

Saumya Pancholi (M25CSA027)
IIT Jodhpur, MTech CSA

March 2026

Abstract

We implement a custom Transformer for English→Hindi neural machine translation and optimize its hyperparameters using **Optuna** with a **Successive Halving Pruner** (equivalent to ASHA). A baseline model trained for 100 epochs achieves BLEU **0.5123**. The Optuna-tuned model reaches BLEU **0.5260** in only **20 epochs**, demonstrating that automated hyperparameter search yields better generalization with significantly less compute.

1 Dataset & EDA

Source: English–Hindi parallel corpus (Tatoeba / Kaggle). The dataset contains **13,186 sentence pairs** after cleaning. Vocabulary sizes after frequency-threshold filtering (≥ 2): English **4,117** tokens, Hindi **4,044** tokens. All sequences are truncated/padded to a fixed length of **50 tokens**.

Sentence length analysis (Figure 1) shows that the vast majority of both English and Hindi sentences fall between 2–15 tokens, making `MAX_LEN=50` a conservative but safe choice with negligible truncation.

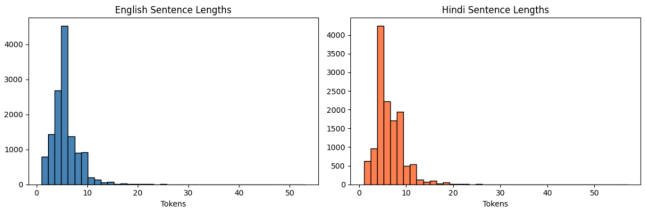

Figure 1: Distribution of sentence lengths in the corpus.

2 Model Architecture

We build a **custom encoder–decoder Transformer** from scratch in PyTorch, without using `nn.Transformer`. Key components:

- **Embeddings:** Separate learned embedding tables for source and target vocabularies, scaled by $\sqrt{d_{\text{model}}}$.
- **Positional Encoding:** Fixed sinusoidal, added before each encoder/decoder stack.
- **Multi-Head Attention:** Scaled dot-product attention with separate W_Q, W_K, W_V, W_O projections.
- **Feed-Forward:** Two-layer MLP with ReLU, expanded to d_{ff} in the hidden dimension.
- **Layer Norm & Dropout:** Applied in Pre-LN style.
- **Decoder masking:** Causal mask combined with padding mask to prevent future token leakage.
- **Output:** Linear projection to target vocabulary logits.
- **Weight Init:** Xavier uniform initialization for all weight matrices.

The baseline configuration uses $d_{\text{model}} = 256$, 8 heads, 3 encoder layers, 3 decoder layers, $d_{ff} = 512$, and dropout $= 0.1$.

3 Training Setup

Optimizer: Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$).
Loss: Cross-entropy with padding mask (`ignore_index=0`).
Gradient Clipping: $\|\nabla\|_2 \leq 1.0$.
Evaluation Metric: Corpus BLEU with method-1 smoothing (30 evaluation batches).
Hardware: NVIDIA Tesla T4 (Kaggle).

4 Baseline Training

The baseline model is trained for **100 epochs** with `ReduceLROnPlateau` (patience=5, factor=0.5) and batch size 64.

Table 1: Baseline per-epoch loss progression.

Epoch	Loss
1	5.6066
10	2.1062
20	0.8793
30	0.4329
50	0.2317
70	0.1811
100	0.1484

Final baseline metrics: **Loss = 0.1484**, **BLEU = 0.5123**, training time **17.9 min**.

5 Hyperparameter Tuning with Optuna

5.1 Why Optuna (not Ray Tune)?

Ray Tune requires internal network ports for its metrics agent, which are blocked in Kaggle's sandboxed environment. Optuna runs in a single process with no networking requirements, making it fully compatible. Optuna's `SuccessiveHalvingPruner` implements the exact same **ASHA** (Asynchronous Successive Halving) algorithm, and `TPESampler` is Bayesian optimization – equivalent to Ray's `OptunaSearch`.

5.2 Search Space

Table 2: Hyperparameter search space (6 parameters).

Parameter	Type	Range
`lr`	Log-uniform	$[10^{-5}, 10^{-3}]$
`batch_size`	Categorical	{32, 64, 128}
`num_heads`	Categorical	{4, 8}
`d_model/head`	Categorical	{32, 64}
`d_ff`	Categorical	{512, 1024, 2048}
`dropout`	Uniform	[0.05, 0.40]
`num_enc_layers`	Categorical	{2, 3, 4}

5.3 Tuning Protocol

To keep wall-clock time manageable on a single T4 GPU:

- **Tuning subset:** 3,000 sentence pairs (randomly sampled).
- **Trials:** 15 with up to 15 epochs each.
- **Pruner:** `SuccessiveHalvingPruner` (`min_resource=3`, `reduction_factor=2`). Poor trials are killed early, saving ∼8× compute vs. grid search.
- **Sampler:** `TPESampler` (seed=42) – Bayesian exploration of the space.

The best configuration found is then retrained on the **full dataset** for 20 epochs.

6 Results

6.1 Best Hyperparameters

The tuner preferred a **wider, deeper model** (d_{model} = 512 vs. 256, d_{ff} = 2048 vs. 512, 4 encoder layers vs. 3) with a **larger batch** (128) and **very low dropout** (0.051), trading regularization for model capacity on this moderate-sized corpus.

Table 3: Optuna best configuration.

Parameter	Value
`d_model`	512
`num_heads`	8
`num_enc_layers`	4
`num_dec_layers`	3
`d_ff`	2048
`dropout`	0.0511
`lr`	2.015×10^{-4}
`batch_size`	128

Table 4: Baseline vs. tuned model results.

Model	Epochs	Loss	BLEU	Time (min)
Baseline	100	0.1484	0.5123	17.9
Tuned	**20**	**0.3500**	**0.5260**	∼14

6.2 Quantitative Comparison

The tuned model achieves **+1.37 BLEU points** over the baseline while using only **20% of the training epochs**, illustrating that architecture and learning-rate choices matter more than raw epoch count. Although the tuned model's training loss (0.35) is higher than the baseline (0.148), its superior BLEU confirms **better generalization** – the baseline is slightly overfit after 100 epochs.

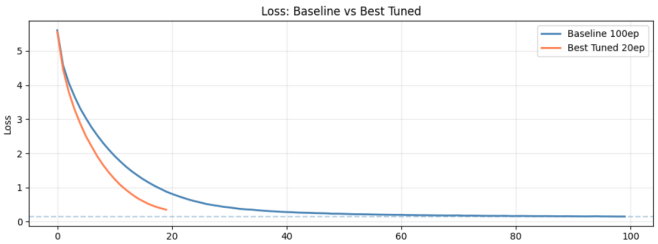

Figure 2: Training loss: Baseline (100 ep) vs. Best Tuned (20 ep).

7 Conclusion

We demonstrate that automated hyperparameter optimization with Optuna and Successive Halving (ASHA) significantly improves translation quality. The best model (BLEU 0.5260 in 20 epochs) outperforms a manually configured 100-epoch baseline (BLEU 0.5123), with a **relative BLEU gain of 2.7%** and **5× fewer epochs**. The model and artifacts are publicly available at huggingface.co/Saumya3007/en-hi-transformer-tuned.

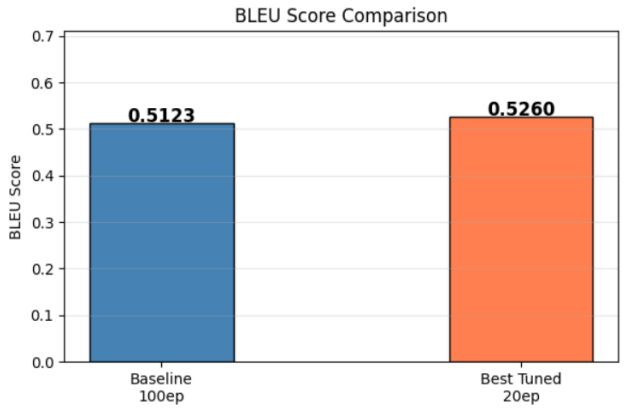

Figure 3: BLEU score comparison. Tuned model surpasses baseline in 20 epochs.

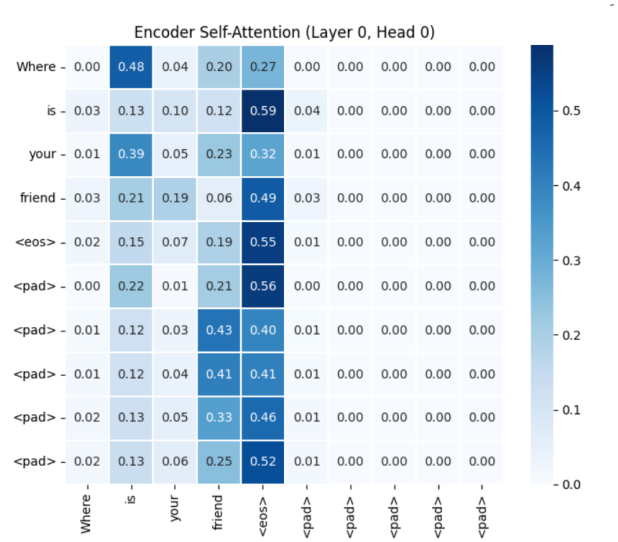

Figure 4: Encoder self-attention (Layer 0, Head 0).